

- **Training data:** Examples and their annotations.
- **Text:** The input text the model should predict a label for.
- **Label:** The label the model should predict.
- **Gradient:** How to change the weights.

Generating a training corpus

```
import spacy

nlp = spacy.blank("en")

# Create a Doc with entity spans
doc1 = nlp("iPhone X is coming")
doc1.ents = [Span(doc1, 0, 2, label="GADGET")]
# Create another doc without entity spans
doc2 = nlp("I need a new phone! Any tips?")

docs = [doc1, doc2]  # and so on...
```

- split data into two portions:
  - **training data:** used to update the model
  - **development data:** used to evaluate the model

```
random.shuffle(docs)
train_docs = docs[:len(docs) // 2]
dev_docs = docs[len(docs) // 2:]
```

- `DocBin` : container to efficiently store and save `Doc` objects
- can be saved to a binary file
- binary files are used for training

```
# Create and save a collection of training docs
train_docbin = DocBin(docs=train_docs)
train_docbin.to_disk("./train.spacy")
# Create and save a collection of evaluation docs
dev_docbin = DocBin(docs=dev_docs)
dev_docbin.to_disk("./dev.spacy")
```

- `spacy convert` lets you convert corpora in common formats
- supports `.conll` , `.conllu` , `.iob` and spaCy's old JSON format

```
$ python -m spacy convert ./train.gold.conll ./corpus
```

INPUT

```
import json
import spacy
from spacy.matcher import Matcher
from spacy.tokens import Span, DocBin

with open("exercises/en/iphone.json", encoding="utf8") as f:
    TEXTS = json.loads(f.read())

nlp = spacy.blank("en")
matcher = Matcher(nlp.vocab)
# Add patterns to the matcher
pattern1 = ([{"LOWER": "iphone"}, {"LOWER": "x"}])
pattern2 = [{"LOWER": "iphone"}, {"IS_DIGIT": True}]
matcher.add("GADGET", [pattern1, pattern2])
docs = []
for doc in nlp.pipe(TEXTS):
    matches = matcher(doc)
    spans = [Span(doc, start, end, label=match_id) for match_id, start, end in matches]
    doc.ents = spans
    docs.append(doc)

doc_bin = DocBin(docs=docs)
doc_bin.to_disk('train.spacy')
```

The training config (1)

- **single source of truth** for all settings
- typically called `config.cfg`
- defines how to initialize the `nlp` object
- includes all settings about the pipeline components and their model implementations
- configures the training process and hyperparameters
- makes your training more reproducible

```
[nlp]
lang = "en"
pipeline = ["tok2vec", "ner"]
batch_size = 1000

[nlp.tokenizer]
@tokenizers = "spacy.Tokenizer.v1"

[components]

[components.ner]
factory = "ner"

[components.ner.model]
@architectures = "spacy.TransitionBasedParser.v2"
hidden_width = 64
# And so on...
```

Generating a config

- spaCy can auto-generate a default config file for you
- interactive quickstart widget in the docs
- `init config` command on the CLI

```
$ python -m spacy init config ./config.cfg --lang en --pipeline ner
```

- `init config` : the command to run
- `config.cfg` : output path for the generated config
- `--lang` : language class of the pipeline, e.g. `en` for English
- `--pipeline` : comma-separated names of components to include

Training a pipeline (1)

- all you need is the `config.cfg` and the training and development data
- config settings can be overwritten on the command line

```
$ python -m spacy train ./config.cfg --output ./output --paths.train train.spacy --paths.dev dev.spacy
```

- `train` : the command to run
- `config.cfg` : the path to the config file
- `--output` : the path to the output directory to save the trained pipeline
- `--paths.train` : override with path to the training data
- `--paths.dev` : override with path to the evaluation data

```
============================ Training pipeline ============================
ℹ Pipeline: ['tok2vec', 'ner']
ℹ Initial learn rate: 0.001

E    #       LOSS TOK2VEC  LOSS NER  ENTS_F  ENTS_P  ENTS_R  SCORE
---  ------  ------------  --------  ------  ------  ------  ------
  0       0          0.00     26.50    0.73    0.39    5.43    0.01
  0     200         33.58    847.68   10.88   44.44    6.20    0.11
  1     400         70.88    267.65   33.50   45.95   26.36    0.33
  2     600         67.56    156.63   45.32   62.16   35.66    0.45
  3     800        138.28    134.12   48.17   74.19   35.66    0.48
  4    1000        177.95    109.77   51.43   66.67   41.86    0.51
  6    1200         94.95     52.13   54.63   67.82   45.74    0.55
  8    1400        126.85     66.19   56.00   65.62   48.84    0.56
 10    1600         38.34     24.16   51.96   70.67   41.09    0.52
 13    1800        105.14     23.23   56.88   69.66   48.06    0.57

✓ Saved pipeline to output directory
/path/to/output/model-last
```

Loading a trained pipeline

- output after training is a regular loadable spaCy pipeline
  - `model-last` : last trained pipeline
  - `model-best` : best trained pipeline

- load it with `spacy.load`

```python
import spacy

nlp = spacy.load("/path/to/output/model-best")
doc = nlp("iPhone 11 vs iPhone 8: What's the difference?")
print(doc.ents)
```

Tip: Packaging your pipeline

- `spacy package` : create an installable Python package containing your pipeline
- easy to version and deploy

```
$ python -m spacy package /path/to/output/model-best ./packages --name my_pipeline --version 1.0.0
```

```
$ cd ./packages/en_my_pipeline-1.0.0
$ pip install dist/en_my_pipeline-1.0.0.tar.gz
```

Load and use the pipeline after installation:

```
nlp = spacy.load("en_my_pipeline")
```

- Existing model can overfit on new data
  - e.g.: if you only update it with `"WEBSITE"`, it can "unlearn" what a `"PERSON"` is

- Also known as "catastrophic forgetting" problem

BAD:

```
LABELS = ["ADULT_SHOES", "CHILDRENS_SHOES", "BANDS_I_LIKE"]
```

GOOD:

```
LABELS = ["CLOTHING", "BAND"]
```